DRiST — Diabetes Risk Screening Tool

Data Preprocessing & Feature Engineering Report

IT2011 — Artificial Intelligence and Machine Learning · Group 2026-Y2-S1-MLB-B12G1-06

Dataset used:
diabetes_binary_5050split_health_indicators_BRFSS2015.csv (Kaggle — Alex Teboul, derived from CDC BRFSS 2015)

Prepared as part of the DRiST proposal's Step 1 (Dataset) and Step 2 (EDA & Balancing) deliverables, plus the feature engineering stage that follows it.

1. Overview

This report documents every step carried out to clean, balance, and engineer features for the diabetes risk dataset, ahead of handing the processed data to the modelling team (Logistic Regression baseline, XGBoost, and FT-Transformer). Each step below records the purpose of the operation, the exact code used, and the measured result from running it against the project dataset.

The pipeline was executed in Google Colab using Python 3, pandas, scikit-learn, seaborn/matplotlib, and joblib. The full notebook is available separately as DRiST_Preprocessing_FeatureEngineering.ipynb.

2. Summary

Stage	Steps	Purpose
Setup	0 – 1	Upload dataset, import libraries
Data understanding	2 – 3	Load data, inspect structure and target column
Data quality	4 – 6	Check missing values, duplicates, valid ranges
EDA	7 – 9	Visualise class balance, correlations, feature spread
Balancing	10 – 12	Merge classes, split train/test, undersample majority class
Feature engineering	13 – 14	Derive new features, encode categories
Finalisation	15 – 16	Scale continuous features, save outputs for the team

3. Step-by-Step Detail

Step 1: Upload the dataset to Colab

Since Google Colab runs on a remote virtual machine, the dataset must be uploaded into the session before it can be read. This step opens a file picker and loads the CSV into Colab's temporary storage.

```
from google.colab import files
uploaded = files.upload()
```

Result

diabetes_binary_5050split_health_indicators_BRFSS2015.csv uploaded and available in the working directory for the rest of the session.

Step 2: Import libraries and set constants

Loads all libraries needed for data handling (pandas, numpy), visualisation (matplotlib, seaborn), preprocessing (scikit-learn), and saving objects (joblib). A fixed RANDOM_STATE is set so every team member gets identical train/test splits and sampling.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import joblib
import os

sns.set_style("whitegrid")
pd.set_option("display.max_columns", 50)
RANDOM_STATE = 42
```

Result

Libraries imported successfully; RANDOM_STATE fixed at 42 for reproducibility across the team.

Step 3: Load the dataset

Reads the uploaded CSV into a pandas DataFrame and confirms its shape (rows × columns).

```
DATA_FILE = "diabetes_binary_5050split_health_indicators_BRFSS2015.csv"
df = pd.read_csv(DATA_FILE)
print("Shape:", df.shape)
df.head()
```

Result

Shape: (70692, 22) — 70,692 survey records across 22 columns (21 predictor features + 1 target column).

Step 4: Inspect structure and identify the target column

Confirms column data types and detects which target column is present — the 3-class Diabetes_012 (raw source file) or the binary Diabetes_binary (pre-balanced file) — so the rest of the pipeline adapts automatically.

```
df.info()

target_col = "Diabetes_012" if "Diabetes_012" in df.columns else "Diabetes_binary"
print("Target column:", target_col)
print(df[target_col].value_counts(normalize=True).round(4) * 100, "%")
```

Result

All 22 columns are numeric (float64), no non-numeric columns found.
Target column detected: Diabetes_binary.
Class balance: 50.0% No Diabetes, 50.0% Diabetes (file is Kaggle's pre-balanced 50/50 split).

Step 5: Check for missing values

Verifies data completeness before any further processing. BRFSS is a cleaned survey export, so no missing values were expected.

```
missing = df.isnull().sum()
print("Total missing values:", missing.sum())
missing[missing > 0]
```

Result

Total missing values: 0 — no imputation required.

Step 6: Check and remove duplicate rows

Identifies exact duplicate survey responses. Even though duplication can legitimately occur (different respondents can answer identically across ~20 binary/ordinal questions), duplicates are removed for modelling so the same response pattern cannot appear in both the training and test split, which would leak information between them.

```
n_dupes = df.duplicated().sum()
print(f"Duplicate rows: {n_dupes} ({n_dupes/len(df):.2%} of data)")
df = df.drop_duplicates().reset_index(drop=True)
print("Shape after dropping duplicates:", df.shape)
```

Result

Duplicate rows found: 1,635 (2.31% of the dataset).
Shape after dropping duplicates: (69057, 22).

Step 7: Sanity-check value ranges

Cross-checks numeric columns against the official BRFSS codebook ranges (e.g. GenHlth must be 1–5, Age bucket 1–13) to catch any invalid or out-of-range survey codes before they reach the model.

```
checks = {
    "BMI": (df["BMI"].between(10, 100)).all(),
    "MentHlth": (df["MentHlth"].between(0, 30)).all(),
    "PhysHlth": (df["PhysHlth"].between(0, 30)).all(),
    "GenHlth": (df["GenHlth"].between(1, 5)).all(),
    "Age": (df["Age"].between(1, 13)).all(),
}
checks
```

Result

All five checked columns passed range validation (BMI, MentHlth, PhysHlth, GenHlth, Age all True) — no invalid survey codes present.

Step 8: EDA — target class distribution

Visualises how many records fall into each class, confirming the balance of the dataset before modelling.

```
plt.figure(figsize=(5,4))
sns.countplot(x=df[target_col])
plt.title("Target class distribution")
plt.xlabel(target_col)
plt.ylabel("Count")
plt.show()
```

Result

Bar chart confirms an approximately even split between the No Diabetes and Diabetes classes after de-duplication (~49.2% / 50.8%).

Step 9: EDA — correlation heatmap

Computes pairwise correlations across all numeric features and ranks which features are most strongly associated with the target, informing which variables are likely to be most predictive.

```
plt.figure(figsize=(12,10))
corr = df.corr(numeric_only=True)
sns.heatmap(corr, cmap="coolwarm", center=0, annot=False)
plt.title("Feature correlation heatmap")
plt.show()

corr[target_col].drop(target_col).sort_values(key=abs, ascending=False)
```

Result

Heatmap generated across all 21 predictor features. GenHlth, HighBP, BMI, DiffWalk, and HighChol emerged as the features most strongly correlated with the diabetes target.

Step 10: EDA — boxplots by class

Compares the distribution of three key indicators (BMI, GenHlth, Age) across the two target classes to visually confirm expected clinical relationships.

```
fig, axes = plt.subplots(1, 3, figsize=(15,4))
sns.boxplot(x=df[target_col], y=df["BMI"], ax=axes[0]); axes[0].set_title("BMI by
class")
sns.boxplot(x=df[target_col], y=df["GenHlth"], ax=axes[1]); axes[1].set_title("GenHlth
by class")
sns.boxplot(x=df[target_col], y=df["Age"], ax=axes[2]); axes[2].set_title("Age bucket by
class")
plt.tight_layout()
plt.show()
```

Result

Boxplots show higher median BMI, poorer self-reported general health, and older age brackets among the Diabetes class, consistent with established clinical risk factors.

Step 11: Merge Prediabetes into Diabetes (conditional)

Where the raw 3-class source file is used, this step merges the Prediabetes class into the Diabetes class to form a binary target, since Prediabetes alone (1.8% of the raw data) is too rare to model reliably on its own. When the pre-balanced binary file is used instead, this step simply confirms the data is already in binary form and skips the merge.

```
if target_col == "Diabetes_012":
    df["Diabetes_binary"] = df["Diabetes_012"].replace({1: 1, 2: 1})
    df = df.drop(columns=["Diabetes_012"])
    target_col = "Diabetes_binary"
else:
    print("Binary file detected — already merged/balanced.")

print(df[target_col].value_counts())
```

Result

Binary file detected — merge step skipped (data already binary).
Diabetes_binary counts: 1.0 → 35,097 records, 0.0 → 33,960 records.

Step 12: Train/test split (before balancing)

Splits the data into training (80%) and test (20%) sets using stratified sampling, so both sets preserve the same class ratio. Splitting happens before any balancing so the test set stays untouched and realistic for final evaluation.

```
feature_cols = [c for c in df.columns if c != target_col]
X = df[feature_cols]
y = df[target_col]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

Result

Train set: (55,245, 21) — Test set: (13,812, 21).
Class balance preserved in both splits (~49.2% / 50.8%).

Step 13: Stratified undersampling (training data only)

Balances the majority class within the training split only, so the model trains on an even class ratio without ever touching or altering the test set. If the data is already balanced (as with the pre-balanced Kaggle file), this step detects that and skips undersampling.

```
train_df = X_train.copy()
train_df[target_col] = y_train.values
class_counts = train_df[target_col].value_counts()
majority_class = class_counts.idxmax()
n_minority = class_counts.min()

if class_counts.min() / class_counts.max() < 0.9:
```

```
    majority_df = train_df[train_df[target_col] == majority_class].sample(
        n=n_minority, random_state=RANDOM_STATE)
    minority_df = train_df[train_df[target_col] != majority_class]
    train_balanced = pd.concat([majority_df, minority_df]).sample(
        frac=1, random_state=RANDOM_STATE).reset_index(drop=True)
else:
    train_balanced = train_df.reset_index(drop=True)

X_train = train_balanced.drop(columns=[target_col])
y_train = train_balanced[target_col]
```

Result

Training data already balanced (ratio above the 0.9 threshold) — undersampling skipped.
Final training class counts: 28,077 Diabetes vs 27,168 No Diabetes.

Step 14: Feature engineering — derive new variables

Creates seven new features from the raw survey indicators to give the models more interpretable, clinically-motivated signals: BMI_Category (WHO weight bands), UnhealthyDays (mental + physical unhealthy days combined), ComorbidityCount (count of major risk conditions), PoorGenHlth (poor self-rated health flag), LifestyleScore (healthy minus risky behaviours), CareAccessGap (insured but skipped care due to cost), and BMI_Age_Interaction (compounding risk of BMI and age).

```
def engineer_features(df_in):
    df_out = df_in.copy()
    df_out["BMI_Category"] = pd.cut(df_out["BMI"],
        bins=[0, 18.5, 25, 30, 100],
        labels=["Underweight","Normal","Overweight","Obese"])
    df_out["UnhealthyDays"] = (df_out["MentHlth"] + df_out["PhysHlth"]).clip(upper=30)
    df_out["ComorbidityCount"] = (df_out["HighBP"] + df_out["HighChol"] +
        df_out["Stroke"] + df_out["HeartDiseaseorAttack"])
    df_out["PoorGenHlth"] = (df_out["GenHlth"] >= 4).astype(int)
    df_out["LifestyleScore"] = (df_out["PhysActivity"] + df_out["Fruits"] +
        df_out["Veggies"] - df_out["HvyAlcoholConsump"])
    df_out["CareAccessGap"] = ((df_out["AnyHealthcare"] == 1) &
        (df_out["NoDocbcCost"] == 1)).astype(int)
    df_out["BMI_Age_Interaction"] = df_out["BMI"] * df_out["Age"]
    return df_out

X_train_fe = engineer_features(X_train)
X_test_fe = engineer_features(X_test)
```

Result

Seven new engineered columns added successfully to both the training and test sets, alongside the original 21 predictors.

Step 15: Encode the categorical feature

One-hot encodes the new BMI_Category column into separate binary columns (all other columns were already numeric). Train and test columns are aligned afterward in case a category is missing from one split.

```
X_train_fe = pd.get_dummies(X_train_fe, columns=["BMI_Category"], drop_first=False)
X_test_fe = pd.get_dummies(X_test_fe, columns=["BMI_Category"], drop_first=False)

X_train_fe, X_test_fe = X_train_fe.align(
    X_test_fe, join="left", axis=1, fill_value=0)

print(X_train_fe.shape, X_test_fe.shape)
```

Result

Final feature count after encoding: 31 columns (21 original + 6 engineered numeric + 4 BMI_Category dummy columns). Train shape: (55245, 31); Test shape: (13812, 31).

Step 16: Scale continuous features

Standardises the continuous columns (BMI, MentHlth, PhysHlth, UnhealthyDays, BMI_Age_Interaction) to mean 0 / standard deviation 1. The scaler is fit on the training data only, then applied to the test data, preventing information leakage from test to train.

```
continuous_cols = ["BMI", "MentHlth", "PhysHlth",
                   "UnhealthyDays", "BMI_Age_Interaction"]

scaler = StandardScaler()
X_train_fe[continuous_cols] = scaler.fit_transform(X_train_fe[continuous_cols])
X_test_fe[continuous_cols] = scaler.transform(X_test_fe[continuous_cols])
```

Result

All five continuous columns rescaled to mean ≈ 0, standard deviation ≈ 1 in the training set; the same fitted transform applied to the test set.

4. Final Dataset Summary

Processed data ready for modelling

Training set: 55,245 rows × 31 features
Test set: 13,812 rows × 31 features
Target: Diabetes_binary (0 = No Diabetes, 1 = Diabetes)

Training class balance: 28,077 Diabetes vs 27,168 No Diabetes (≈50/50)
New engineered features: BMI_Category (4 dummy columns), UnhealthyDays, ComorbidityCount, PoorGenHlth, LifestyleScore, CareAccessGap, BMI_Age_Interaction
Scaling: StandardScaler fit on training data only, applied identically to test data
No missing values; 1,635 duplicate rows removed prior to splitting

5. Handoff Notes

- Teammates working on Logistic Regression, XGBoost, and the FT-Transformer should load processed_data/X_train.csv, X_test.csv, y_train.csv, and y_test.csv directly, rather than re-running preprocessing.
- processed_data/scaler.pkl should be reused (not refit) if any new/unseen data needs to be scaled later, e.g. for a live demo or additional test data.
- RANDOM_STATE = 42 was used throughout so results are reproducible across the team.
- If the team later switches to the raw 3-class file (diabetes_012_health_indicators_BRFSS2015.csv) to perform the merge and undersampling manually as originally proposed, Steps 10–12 already contain the conditional logic to handle that file without any code changes.

Newly Added features details.

1. **BMI_Category**

   What it does: Converts raw BMI (a number like 26.4) into WHO's standard weight categories.

   Why: A raw BMI number is continuous and harder for some models — and for humans reading your report — to interpret directly. Doctors don't think in "BMI = 31.2," they think "obese." This gives the model a feature that mirrors how clinicians actually assess risk, and it can catch non-linear effects (e.g. risk doesn't rise smoothly with BMI — it jumps once you cross into "Obese").

```
df_out["BMI_Category"] = pd.cut(df_out["BMI"], bins=[0, 18.5, 25, 30, 100],
  labels=["Underweight", "Normal", "Overweight", "Obese"])
```

2. **UnhealthyDays**

   What it does: Adds mental-health bad days + physical-health bad days (both already 0–30 scales in the raw data) into one combined score, capped at 30.

   Why: The original dataset has these as two separate columns. Combined, they give a single "overall poor health" signal — someone struggling with 15 bad mental days AND 20 bad physical days is a stronger risk signal than either number alone suggests.

```
df_out["UnhealthyDays"] = (df_out["MentHlth"] + df_out["PhysHlth"]).clip(upper=30)
```

3.ComorbidityCount

What it does: Counts how many of these 4 major conditions (high blood pressure, high cholesterol, stroke, heart disease) a person has, from 0–4.

Why: Diabetes risk isn't just about having one condition — it's about how many risk factors stack up together. A count (0, 1, 2, 3, or 4) captures "cumulative risk burden" in a way that's more informative than four separate yes/no flags scattered across the dataset.

```
df_out["ComorbidityCount"] = (df_out["HighBP"] + df_out["HighChol"] +
   df_out["Stroke"] + df_out["HeartDiseaseorAttack"])
```

4.PoorGenHlth

What it does: Turns the 5-point self-rated health scale (1=Excellent → 5=Poor) into a simple flag: 1 if someone rated their health "Poor" or "Fair", 0 otherwise.

Why: Simplifies a subjective 5-level scale into a clear binary "at risk" signal, which is easier to interpret directly, and highlights the group most worth flagging for screening.

```
df_out["PoorGenHlth"] = (df_out["GenHlth"] >= 4).astype(int)
```

5.LifestyleScore

What it does: Adds up healthy behaviors (exercise, eating fruit, eating vegetables) and subtracts a risky one (heavy alcohol use), giving a rough score from -1 to 3.

Why: Instead of the model having to learn on its own that , "exercise + fruit + veggies - alcohol = healthier lifestyle," we hand it that combined signal directly as one number.

```
df_out["LifestyleScore"] = (df_out["PhysActivity"] + df_out["Fruits"] +
   df_out["Veggies"] - df_out["HvyAlcoholConsump"])
```

6. CareAccessGap

What it does: Flags people who *have* health insurance but *still* skipped seeing a doctor because of cost.

Why: This is a specific, meaningful pattern — it's not just "no insurance," it's "insurance isn't actually helping this person get care." That's a distinct group who may have undiagnosed conditions specifically because of this gap, which raw individual columns don't capture on their own.

```
df_out["CareAccessGap"] = ((df_out["AnyHealthcare"] == 1) &
   (df_out["NoDocbcCost"] == 1)).astype(int)
```

7.BMI_Age_Interaction

What it does: Multiplies BMI by age bucket.

Why: Diabetes risk from high BMI compounds with age — a high BMI at age 25 carries different risk than the same BMI at age 60. Multiplying the two lets the model see that combined effect directly, instead of treating BMI and age as two totally unrelated numbers.

```
df_out["BMI_Age_Interaction"] = df_out["BMI"] * df_out["Age"]
```